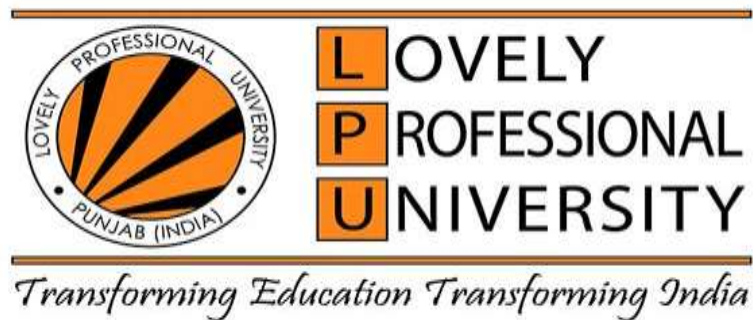




A Project on

KEYLOGGER & COUNTERMEASURE DEMONSTRATOR

Submitted by:

Navpreet Kaur Dhillon- R9PV45A22

Sadia Naseer- R9PV45A39

Vusumuzi Zondo- R9PV45A11

Sushanti Anam- R9PV45A46

Under the Supervision of:

Mr. Manish Singh

Assistant Professor

Cyber Security and Network

Submitted to:

Lovely Professional University, Punjab

in partial fulfilment of the requirement for the award of

BSc. Information Technology

Batch 2024-2027

DECLARATION

I, Navpreet Kaur Dhillon, Sadia Naseer, Vusumuzi Zondo, Sushanti Anam, hereby declare that the Project Report titled- Keylogger & Countermeasure Demonstrator submitted to- Lovely Professional University, Punjab in partial fulfilment of the requirement for the award of the degree BSc. Information Technology is my original work. The findings and conclusions are based on data collected and analyzed by me and have not been submitted elsewhere for any degree or diploma.

Date: 07-07-2026

Name of the student: Navpreet Kaur Dhillon, Sadia Naseer, Vusumuzi Zondo, Sushanti Anam.

Roll no: R9PV45A22, R9PV45A39, R9PV45A11, R9PV45A46

ACKNOWLEDGEMENTS

The successful completion of this project would not have been possible without the support, guidance, and encouragement of several individuals, to whom I express my sincere gratitude.

I would like to thank my project supervisor, Mr. Manish Singh, for providing valuable guidance, constructive suggestions, and continuous encouragement throughout the development of this project. His expertise and insightful feedback greatly contributed to the successful completion of the work.

I also wish to thank my group members for their teamwork, discussions, and support during the course of this project. Their suggestions and cooperation proved valuable at various stages of development.

I sincerely acknowledge everyone who directly or indirectly contributed to the successful completion of this project.

Name of the student- Navpreet Kaur Dhillon, Sadia Naseer, Vuzumuzi Zondo, Sushanti Anam

TABLE OF CONTENTS

Serial no.	Content	Page no.
1	STUDENT'S DECLARATION	2
2	ACKNOWLEDGEMENT	2
3	ABSTRACT	5
CH-1	INTRODUCTION	6
<i>1.1</i>	<i>Introduction</i>	6
<i>1.2</i>	<i>Rationale Of The Study</i>	7
<i>1.3</i>	<i>Overview Of The Project</i>	8
<i>1.4</i>	<i>Literature Review</i>	9
CH-2	OBJECTIVES OF THE STUDY	9
<i>2.1</i>	<i>Objectives</i>	9
<i>2.2</i>	<i>Primary Objective</i>	10
<i>2.3</i>	<i>Specific Objectives</i>	10
<i>2.4</i>	<i>Project Outcomes</i>	11
<i>2.5</i>	<i>Significance Of The Study</i>	12
CH-3	PROBLEM STATEMENT	13
<i>3.1</i>	<i>Problem Statement</i>	13
<i>3.2</i>	<i>Problem Identification</i>	14
<i>3.3</i>	<i>Proposed System</i>	14
<i>3.4</i>	<i>Advantages Of The Proposed System</i>	15
CH-4	METHODOLOGY	16
<i>4.1</i>	<i>Methodology</i>	16
<i>4.2</i>	<i>Development Methodology</i>	16
<i>4.3</i>	<i>Software Requirement Specification</i>	18

<i>4.4</i>	<i>Software Architecture</i>	19
<i>4.5</i>	<i>Functional Modules</i>	19
<i>4.6</i>	<i>Database Design</i>	24
<i>4.7</i>	<i>Technology Stack</i>	24
<i>4.8</i>	<i>System Workflow</i>	25
<i>4.9</i>	<i>Advantages Of The Methodology</i>	28
CH-5	DATA ANALYSIS & INTERPRETATION	28
<i>5.1</i>	<i>Data Analysis & Interpretation</i>	28
<i>5.2</i>	<i>Experimental Setup</i>	29
<i>5.3</i>	<i>Module-Wise Testing</i>	30
<i>5.4</i>	<i>Screenshots Of Implementation</i>	33
<i>5.5</i>	<i>Sample Output</i>	34
<i>5.6</i>	<i>Functional Analysis</i>	34
<i>5.7</i>	<i>Performance Analysis</i>	35
<i>5.8</i>	<i>Advantages Observed</i>	35
<i>5.9</i>	<i>Limitations Observed</i>	36
<i>5.10</i>	<i>Objective Achievement Analysis</i>	36
<i>5.11</i>	<i>Test Case Summary</i>	37
<i>5.12</i>	<i>Interpretation Of Results</i>	37
CH-6	DISCUSSION AND RESULTS	37
<i>6.1</i>	<i>Discussion And Results</i>	37
<i>6.2</i>	<i>Discussion Of Project Outcomes</i>	38
<i>6.3</i>	<i>Analysis Of Individual Modules</i>	38
<i>6.4</i>	<i>Achievement Of Project Objectives</i>	40
<i>6.5</i>	<i>Discussion From A Cybersecurity Perspective</i>	40
<i>6.6</i>	<i>Strengths Of The Proposed System</i>	41
<i>6.7</i>	<i>Limitations Of The Developed System</i>	41
<i>6.8</i>	<i>Comparison With Existing Educational Approaches</i>	42

<i>6.9</i>	<i>Practical Applications</i>	43
<i>6.10</i>	<i>Future Improvements</i>	43
<i>6.11</i>	<i>Overall Evaluation</i>	44
CH-7	CONCLUSION	44
CH-8	RECOMMENDATIONS & FUTURE SCOPE	45
<i>8.1</i>	<i>Recommendations & Future Scope</i>	45
<i>8.2</i>	<i>Technical Recommendations</i>	46
<i>8.3</i>	<i>Security Recommendations</i>	47
<i>8.4</i>	<i>Countermeasure Improvements</i>	48
<i>8.5</i>	<i>Artificial Intelligence Integration</i>	48
<i>8.6</i>	<i>Cloud Integration</i>	49
<i>8.7</i>	<i>User Interface Enhancements</i>	49
<i>8.8</i>	<i>Performance Optimization</i>	50
<i>8.9</i>	<i>Research Opportunities</i>	50
<i>8.10</i>	<i>Industrial Applications</i>	51
<i>8.11</i>	<i>Overall Recommendations</i>	51
<i>8.12</i>	<i>Future Scope</i>	52
	APPENDIX	53
	BIBLIOGRAPHY	60

ABSTRACT

This project presents the design and implementation of an educational cybersecurity application titled "**Keylogger & Countermeasure Demonstrator.**" The objective of the project is to provide students with a practical understanding of keyboard event monitoring and basic defensive concepts through a controlled and ethical demonstration. The application has been developed using Python and integrates the pynput, Flask, SQLite, and Tkinter libraries to create an interactive learning platform.

The system is composed of multiple functional modules that work together to demonstrate the flow of keyboard data within a local environment. A keyboard event listener records user keystrokes and stores them in an SQLite database along with corresponding timestamps. A Flask-based web dashboard retrieves the stored records and presents them in a structured format, allowing users to observe how logged information is maintained and displayed. In addition, a Tkinter-based virtual keyboard has been incorporated to demonstrate an alternative text-input interface as part of the project's educational countermeasure component.

A modular development approach was adopted to simplify implementation and improve maintainability. Individual modules responsible for keyboard event handling, database operations, web-based visualization, and graphical user interaction were designed, tested, and integrated to form the complete application. Functional testing confirmed the successful operation of each module, including data capture, database management, dashboard rendering, and graphical interface functionality.

The completed system demonstrates the practical application of Python in cybersecurity while reinforcing concepts related to endpoint security, secure software development, database integration, and graphical interface design. Although the application is intended solely for educational use and operates only within a local environment, it provides a strong foundation for understanding the techniques involved in keyboard event monitoring and highlights the importance of ethical cybersecurity practices.

Future enhancements may include behavioural process monitoring, intelligent anomaly detection, encrypted database storage, authentication mechanisms, and integration with advanced endpoint security solutions. These improvements would further extend the educational value and practical relevance of the project.

CHAPTER 1

1.1 INTRODUCTION

With the rapid growth of digital communication and online services, protecting sensitive user information has become one of the primary concerns in cybersecurity. Every day, users enter confidential information such as passwords, banking credentials, personal messages, and authentication codes through computer keyboards. This sensitive information is often targeted by malicious software known as keyloggers, which secretly record keystrokes without the

user's knowledge. Once captured, the recorded data can be exploited by attackers for identity theft, financial fraud, unauthorized system access, and various cybercrimes.

A keylogger is a type of monitoring software designed to intercept and record keyboard input. While keyloggers have legitimate applications in areas such as parental monitoring, employee supervision (with proper authorization), accessibility research, and cybersecurity education, they are more commonly associated with malicious activities. Software-based keyloggers operate by interacting with the operating system's input-handling mechanisms, enabling them to capture user keystrokes in real time. This makes them one of the most commonly used techniques for credential theft and information gathering.

Understanding how keyloggers function is essential for cybersecurity professionals because awareness of attack techniques contributes to the development of effective defensive strategies. Educational demonstrations provide students and researchers with practical insight into the operation of such software in controlled environments while emphasizing the ethical and legal responsibilities associated with cybersecurity research.

The project "Keylogger & Countermeasure Demonstrator" has been developed as an educational application to demonstrate both the offensive concept of keystroke logging and a corresponding defensive mechanism. The implementation consists of a Python-based application that records keyboard input using the pynput library and stores the captured keystrokes in an SQLite database. A local Flask-based dashboard displays the recorded entries, illustrating how collected information could be organized and reviewed in a controlled demonstration. In addition, the project includes a Tkinter-based virtual keyboard that demonstrates an alternative text-entry method intended to reduce reliance on conventional keyboard input for educational discussion of defensive techniques.

The project has been developed exclusively for educational purposes on isolated local systems. It is intended to increase awareness of endpoint security concepts, demonstrate the importance of secure input mechanisms, and encourage the study of defensive cybersecurity practices.

1.2 RATIONALE OF THE STUDY

The primary motivation behind this project is to enhance cybersecurity awareness through practical demonstration. Although keyloggers are frequently discussed in cybersecurity

courses, many students possess only theoretical knowledge regarding their operation. Observing how keyboard events are captured, stored, and displayed provides a clearer understanding of the potential risks associated with keystroke logging.

The project also emphasizes the importance of defensive thinking. Rather than focusing solely on offensive techniques, it introduces a simple countermeasure using a virtual keyboard interface. This allows students to appreciate how alternative input methods can contribute to reducing exposure to traditional keyboard interception techniques in specific scenarios.

Furthermore, the project demonstrates the integration of multiple technologies—including Python, Flask, SQLite, Tkinter, threading, and pynput—within a single educational application. This multidisciplinary approach provides practical experience in software development while reinforcing cybersecurity concepts.

1.3 OVERVIEW OF THE PROJECT

The Keylogger & Countermeasure Demonstrator is a Python-based educational application consisting of three primary modules.

The first module captures keyboard events and records them into a local SQLite database. Keyboard events are processed through a listener implemented using the pynput library, while timestamps are recorded to maintain an ordered history of captured input.

The second module consists of a Flask web application that retrieves stored keystroke records from the database and presents them through a local dashboard. The dashboard refreshes automatically at regular intervals, allowing users to observe newly stored records during the demonstration.

The third module provides a Tkinter-based virtual keyboard that enables users to enter characters using an on-screen interface. This module demonstrates an educational countermeasure by illustrating an alternative input mechanism within the project environment.

Together, these modules demonstrate the concepts of keystroke collection, secure local storage, information visualization, and defensive awareness within a controlled educational setting.

1.4 LITERATURE REVIEW

Cybersecurity researchers have extensively studied keylogging techniques due to their widespread use in credential theft and information harvesting. Software keyloggers typically rely on operating system APIs, keyboard hooks, or event listeners to capture user input before applications process the keystrokes. These techniques have been documented in malware analysis research and endpoint security literature.

The SANS Institute emphasizes that endpoint protection should focus not only on signature-based detection but also on behavioral monitoring capable of identifying suspicious software activities. Modern endpoint security products combine process monitoring, file integrity analysis, memory inspection, and anomaly detection to identify malicious software before significant damage occurs.

The Python pynput library is widely used for legitimate automation, accessibility tools, testing, and educational research because it provides access to keyboard and mouse events across multiple operating systems. Within this project, pynput serves as an educational example illustrating how keyboard events may be observed in a controlled environment.

Flask has become one of the most popular lightweight Python web frameworks for rapidly developing web applications. Its simplicity makes it suitable for constructing demonstration dashboards, allowing recorded information to be presented through a browser interface. SQLite complements Flask by providing a lightweight relational database that requires minimal configuration while supporting reliable storage of timestamped records.

Tkinter, Python's standard graphical user interface toolkit, provides an effective platform for building educational desktop applications. In this project, it is used to implement a virtual keyboard that demonstrates an alternative method of user interaction within the educational environment.

CHAPTER 2

2.1 OBJECTIVES OF THE STUDY

Every cybersecurity project is developed with specific goals that address an identified security problem while contributing to knowledge, awareness, or technological advancement. The Keylogger & Countermeasure Demonstrator has been designed as an educational cybersecurity project to provide practical insight into how software-based keyloggers function and how defensive techniques can be employed to improve endpoint security awareness.

While keylogging is commonly associated with malicious software, understanding its operational principles is essential for cybersecurity students, security analysts, and researchers. This project provides a controlled environment in which users can observe the process of keyboard event capture, secure local data storage, and visualization through a monitoring dashboard. At the same time, it introduces a defensive component through a virtual keyboard interface that demonstrates an alternative method of text input for educational discussion.

The objectives presented in this chapter define the direction of the project and establish measurable outcomes that align with the project's educational purpose.

2.2 PRIMARY OBJECTIVE

The primary objective of this project is:

To design and develop an educational software application that demonstrates the concept of software-based keystroke logging and introduces a corresponding defensive mechanism to enhance cybersecurity awareness and understanding of endpoint security concepts.

This objective focuses on enabling students and researchers to understand how keyboard events can be captured, stored, and visualized within a controlled local environment while emphasizing ethical cybersecurity practices.

2.3 SPECIFIC OBJECTIVE

To accomplish the primary objective, the following specific objectives have been identified:

Objective 1

To develop a Python-based educational application capable of demonstrating keyboard event capture within a controlled local environment using the pynput library.

Objective 2

To securely store captured keyboard events in a lightweight SQLite database together with timestamps for organized record management and retrieval.

Objective 3

To develop a Flask-based web dashboard that retrieves stored records from the database and presents them through an organized monitoring interface for educational visualization.

Objective 4

To design a Tkinter-based virtual keyboard that demonstrates an alternative text-entry mechanism as part of the project's educational countermeasure component.

Objective 5

To demonstrate the interaction between multiple Python technologies—including Flask, SQLite, threading, Tkinter, and pynput—within a single cybersecurity application.

Objective 6

To improve understanding of endpoint security concepts by illustrating how keyboard input can be monitored and why defensive awareness is necessary for protecting sensitive information.

Objective 7

To provide students with practical exposure to secure software development concepts through an educational demonstration implemented entirely on an isolated local system.

2.4 PROJECT OUTCOMES

The successful completion of this project is expected to achieve the following outcomes.

Outcome 1

Students will understand the basic working principles of software-based keyloggers and their role in cybersecurity.

Outcome 2

Students will gain practical knowledge of keyboard event handling using Python libraries within a controlled educational environment

Outcome 3

Students will understand how captured information can be securely stored and retrieved using relational database systems such as SQLite.

Outcome 4

Students will learn how Flask can be integrated with backend data storage to create a real-time monitoring dashboard.

Outcome 5

Students will gain familiarity with graphical user interface development using Tkinter through the implementation of an educational virtual keyboard.

Outcome 6

Students will develop awareness regarding endpoint security and the importance of protecting user credentials from unauthorized monitoring.

Outcome 7

Students will understand the ethical responsibilities associated with cybersecurity research and the importance of conducting security demonstrations only in authorized and isolated environments.

2.5 SIGNIFICANCE OF THE STUDY

The significance of this project extends across multiple areas of cybersecurity education.

Academic Significance

The project provides practical learning opportunities for students studying cybersecurity, computer science, and information technology by combining theoretical concepts with software implementation.

Technical Significance

The project demonstrates the integration of several widely used Python technologies within a single application, including Flask for web development, SQLite for database management, Tkinter for desktop interfaces, threading for concurrent execution, and pynput for educational keyboard event handling.

Security Significance

The project increases awareness regarding endpoint security by illustrating how keyboard input may be monitored and emphasizing the importance of secure computing practices.

Educational Significance

The project serves as a laboratory demonstration that allows instructors and students to visualize cybersecurity concepts that are often difficult to understand through theory alone.

CHAPTER 3

3.1 PROBLEM STATEMENT

The rapid advancement of information technology has significantly increased the amount of sensitive information processed through digital devices. Users routinely enter confidential information such as usernames, passwords, banking credentials, personal identification numbers (PINs), confidential emails, and organizational data using computer keyboards. Protecting this information has become a major challenge due to the growing sophistication of cyber threats.

Among various credential-stealing techniques, software-based keyloggers remain one of the most effective tools used by cybercriminals. Unlike many traditional attacks that exploit software vulnerabilities, keyloggers target the human interaction layer by silently recording

keyboard activity before information can be encrypted or securely transmitted. This makes keylogging a significant endpoint security concern for both individuals and organizations.

Although antivirus software and endpoint protection systems have evolved considerably, understanding how keyloggers function remains an essential part of cybersecurity education. Students often learn about these attacks theoretically but rarely gain practical insight into how keyboard events are captured, stored, and analyzed. This gap between theoretical knowledge and practical understanding forms the basis of the present study.

3.2 PROBLEM IDENTIFICATION

Many computer users believe that installing antivirus software is sufficient to protect their systems from credential theft. However, attackers continue to develop techniques that target user input directly rather than exploiting operating system vulnerabilities.

Keyloggers present several security challenges:

- Users are generally unaware that their keyboard activity may be monitored.
- Sensitive credentials can be intercepted before encryption takes place.
- Traditional users often fail to recognize the signs of endpoint monitoring.
- Lack of awareness increases the likelihood of successful credential theft.
- Students studying cybersecurity frequently lack practical exposure to endpoint attack techniques.

These challenges demonstrate the need for educational projects that safely illustrate how keyboard-event monitoring operates while reinforcing the importance of cybersecurity awareness.

3.3 PROPOSED SYSTEM

The proposed Keylogger & Countermeasure Demonstrator has been developed to address these educational limitations.

The system integrates three major functional modules.

Module 1 – Keyboard Event Demonstration

The first module demonstrates keyboard-event capture using Python's pynput library. Captured events are converted into readable values and passed to the database module for storage.

Module 2 – Database Management

The second module stores timestamped keyboard events in an SQLite database. The database automatically initializes itself when the application starts and maintains a structured history of recorded entries.

Module 3 – Monitoring Dashboard

The third module uses the Flask framework to retrieve stored records and display them through a local web dashboard. The interface refreshes automatically, enabling users to observe newly recorded entries during the demonstration.

Module 4 – Countermeasure Demonstration

A Tkinter-based virtual keyboard provides an educational demonstration of an alternative text-entry mechanism. Instead of relying solely on physical keyboard input, users can enter characters through an on-screen keyboard interface designed for awareness purposes.

3.4 ADVANTAGES OF THE PROPOSED SYSTEM

The proposed system offers several educational and technical advantages.

- Provides practical understanding of keyboard-event handling.
- Demonstrates secure local storage using SQLite.
- Integrates desktop and web technologies into one application.
- Visualizes stored information using a Flask dashboard.
- Encourages cybersecurity awareness through practical learning.
- Demonstrates an educational defensive interface.
- Operates entirely within a local environment.

- Uses open-source technologies that are easy to understand and modify.
-

CHAPTER 4

4.1 METHODOLOGY

Methodology refers to the systematic process adopted for the design, development, implementation, and evaluation of a software system. It explains the techniques, technologies, workflow, and architectural decisions involved in developing the proposed solution. A well-defined methodology ensures that every component of the project is logically organized and contributes toward achieving the intended objectives.

The Keylogger & Countermeasure Demonstrator has been developed using Python and several of its supporting libraries to demonstrate both keyboard event monitoring and an educational defensive mechanism. The project integrates a keyboard event listener, a lightweight relational database, a web-based monitoring dashboard, and a desktop graphical user interface. These components work together within a local environment to provide an educational demonstration of endpoint security concepts.

4.2 DEVELOPMENT METHODOLOGY

The project follows a simplified Waterfall Software Development Model, where each stage is completed sequentially before proceeding to the next phase.

Phase 1 – Requirement Analysis

During this phase, the project objectives and educational requirements were identified. The system was designed to demonstrate keyboard event handling while incorporating a simple defensive interface.

Phase 2 – System Design

The architecture of the project was designed by separating the application into independent modules:

- Keyboard Event Module
- Database Module
- Web Dashboard
- Countermeasure Module

This modular architecture improves readability, maintainability, and future scalability.

Phase 3 – Implementation

The system was implemented using Python.

Different Python libraries were integrated to perform different tasks.

- pynput
- Flask
- SQLite
- Tkinter
- Threading

Phase 4 – Testing

Each module was tested individually before integrating the complete application.

Testing focused on-

- Keyboard input
- Database insertion
- Dashboard visualization
- Virtual keyboard functionality

Phase 5 – Evaluation

The completed application was evaluated by observing whether

- keyboard events were recorded,
- database entries were created,
- dashboard displayed records correctly,

- virtual keyboard operated successfully.

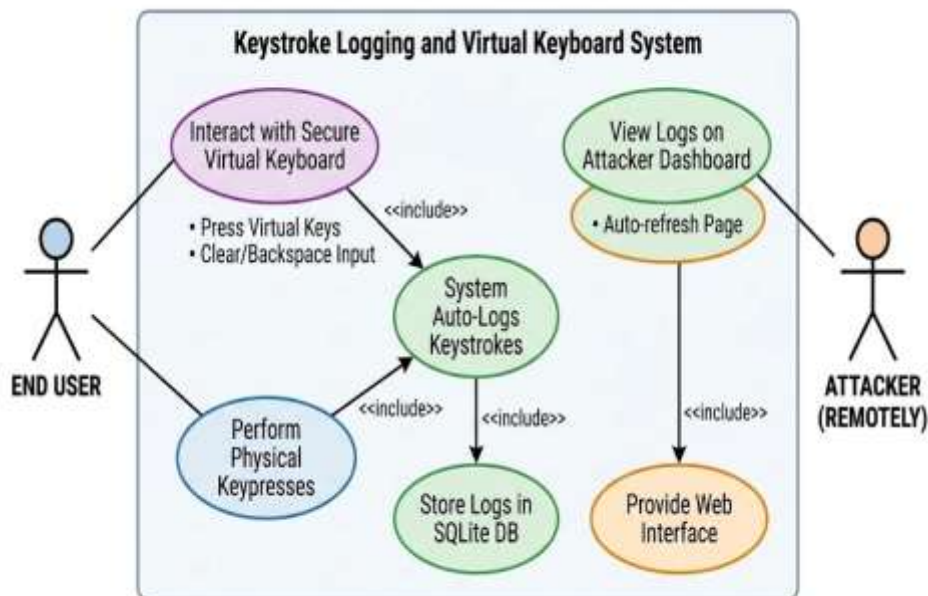


Figure 4.1: Use Case Diagram of the Keylogger & Countermeasure Demonstrator

4.3 SOFTWARE REQUIREMENT SPECIFICATION

Operating System

- Windows 10 / Windows 11

Programming Language

- Python 3.x

Python Libraries

- Flask
- pynput
- sqlite3
- Tkinter
- threading

4.4 SOFTWARE ARCHITECTURE

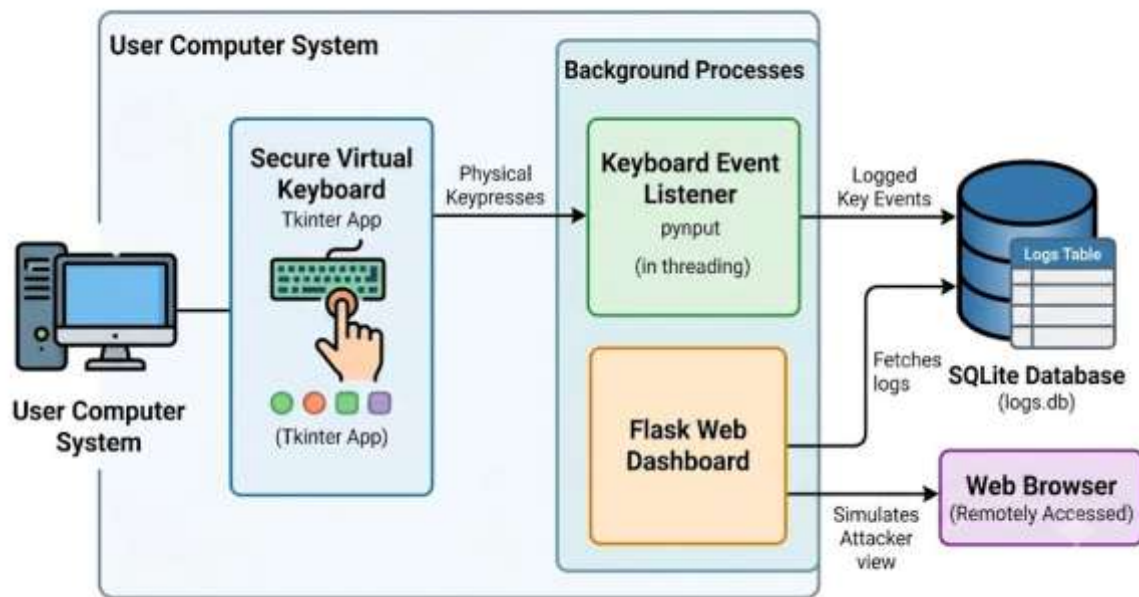


Figure 4.2: Overall System Architecture of the Keylogger & Countermeasure Demonstrator

4.5 FUNCTIONAL MODULES

The project has been divided into three major implementation modules.

Main Application Module (app.py)

The main application serves as the central controller of the project.

It integrates

- keyboard event handling,
- Flask,
- threading,
- database communication.

When executed, the application creates a background thread responsible for keyboard event monitoring while simultaneously launching the Flask web server. This allows both services to operate concurrently without interrupting each other.

Major responsibilities include:

- Starting keyboard listener
- Capturing keyboard events
- Forwarding events to database
- Launching Flask server
- Displaying monitoring dashboard

Working Principle

Application Starts



Background Thread Created



Keyboard Listener Activated



Flask Server Starts



Dashboard Ready

Keyboard Event Capture Module

The keyboard event listener is implemented using Python's pynput library.

Whenever the user presses a key, the listener detects the event, identifies the corresponding key, converts special keys into readable text, and forwards the information to the database module.

Examples include-

Space

[SPACE]

Enter

[ENTER]

Other keys

A

B

C

Special keys

[SHIFT]

[CTRL]

[ALT]

This conversion improves readability of stored records.

Database Module (database.py)

The database module manages all storage operations.

SQLite has been selected because

- lightweight
- serverless
- easy deployment
- no additional configuration

Upon execution, the application automatically creates the database if it does not already exist.

The module provides-

- database initialization,
- record insertion,

- record retrieval.

Each record contains-

- timestamp
- captured key

Database Schema

Table: keystrokes

FIELD	DATATYPE	DESCRIPTION
id	Integer	Primary key
timestamp	Text	Date and time
Key_pressed	Text	Captured

Database Workflow

Keyboard Event

|



save_key()

|



SQLite

|



Stored Record

Flask Dashboard Module

The Flask framework provides a lightweight web interface that displays stored keyboard records.

The dashboard retrieves database entries using

get_all_keys()

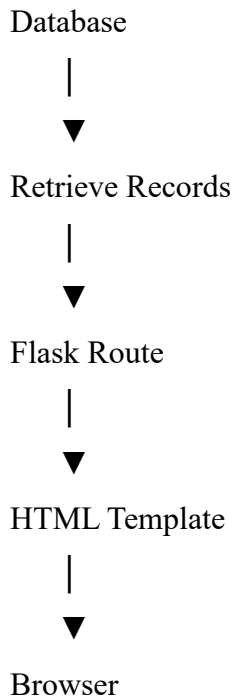
and displays

- Timestamp

- Captured Key

The webpage automatically refreshes every three seconds, ensuring that newly stored entries become visible without manual intervention.

Dashboard Workflow



Countermeasure Module (defence.py)

The countermeasure module has been developed using Tkinter.

Instead of relying solely on the physical keyboard, users interact with an on-screen QWERTY keyboard that displays typed characters within the application window. The interface includes commonly used keys such as letters, numbers, Space, Backspace, and Clear.

The window remains on top of other applications to improve accessibility during demonstrations.

Functional Components-

- Display Box
- QWERTY Layout
- Space Key
- Backspace Key

- Clear Key
- Button-based Input

4.6 DATABASE DESIGN

SQLite stores all keyboard records.

Advantages-

- Lightweight
- Fast
- Portable
- No installation
- Suitable for educational projects

The database consists of only one table,
making the design simple and easy to understand.

4.7 TECHNOLOGY STACK

Technology	Purpose
Python	Core Programming Language
pynput	Keyboard Event Handling
SQLite	Local Database
Flask	Web Dashboard
Tkinter	Desktop GUI

4.8 SYSTEM WORKFLOW

Overall project workflow

Application Starts



Database Initialized



Keyboard Listener Starts



Key Press Detected



Database Storage



Flask Dashboard Updates



User Views Logs

Countermeasure

User



Virtual Keyboard



Display Box

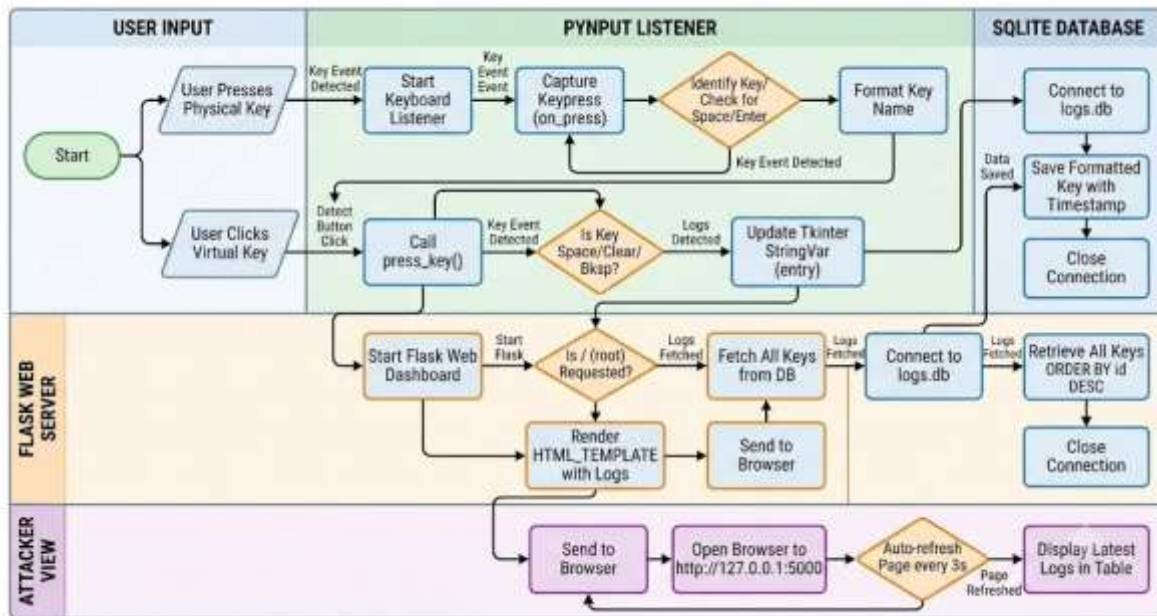


Figure 4.3: System Flowchart Showing the Overall Execution Process

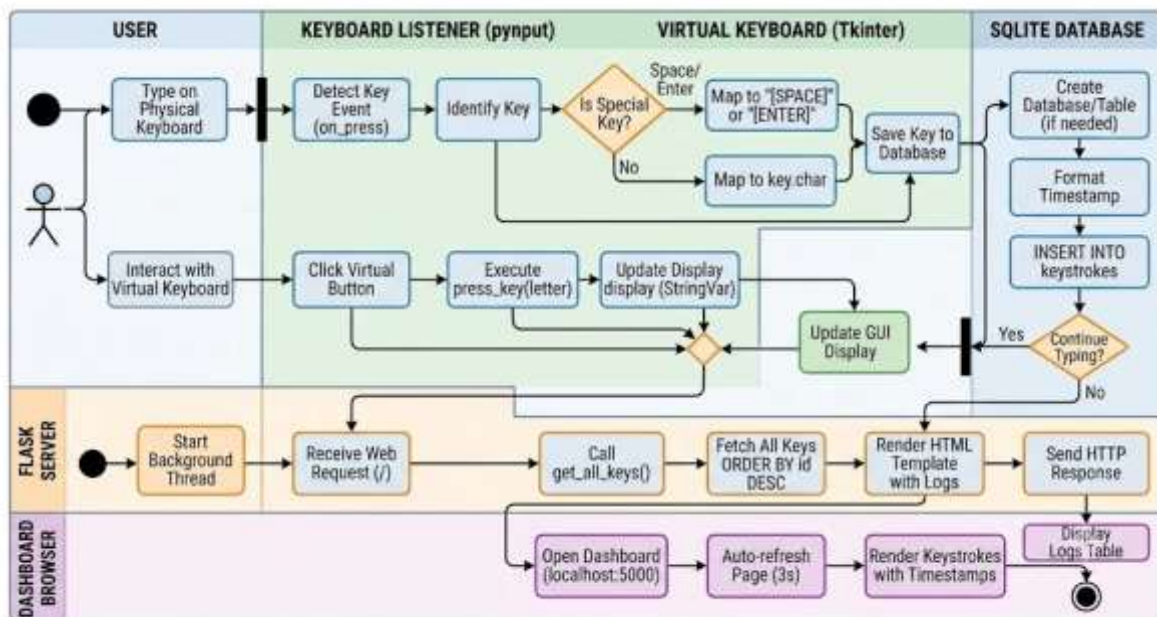


Figure 4.4: Activity Diagram of the Keystroke Logging and Monitoring Process

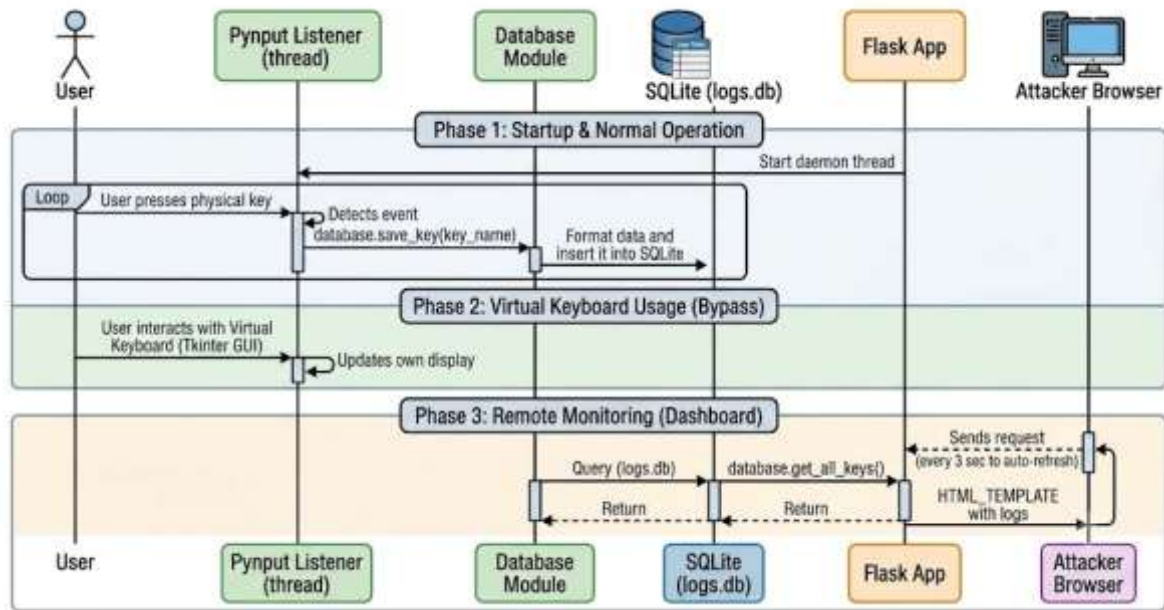


Figure 4.5: Sequence Diagram Showing Communication Among System Components

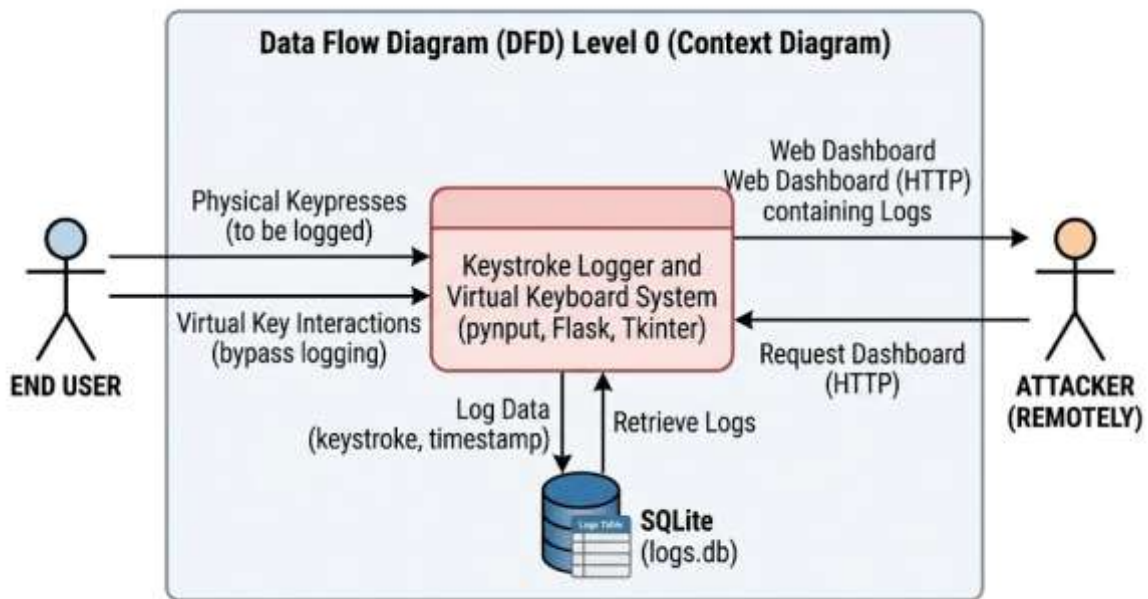


Figure 4.6: Data Flow Diagram (Level 0) of the Proposed System

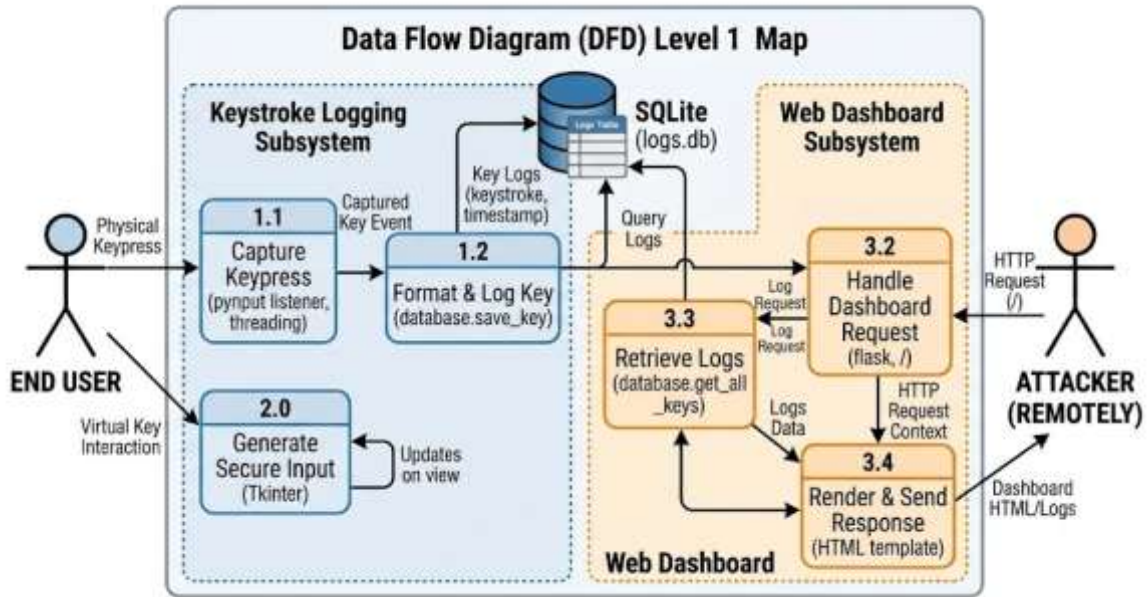


Figure 4.7: Data Flow Diagram (Level 1) of the Proposed System

4.9 ADVANTAGES OF THE METHODOLOGY

The adopted methodology offers several advantages:

- Modular implementation.
- Easy maintenance.
- Lightweight database management.
- Concurrent execution through threading.
- Simple deployment.
- Clear separation of responsibilities.
- Easy scalability for future enhancements.

CHAPTER 5

5.1 DATA ANALYSIS & INTERPRETATION

This chapter presents the implementation results and analysis of the Keylogger & Countermeasure Demonstrator. The purpose of this chapter is to evaluate the functionality of each module, observe system behavior during execution, and interpret the outputs generated by the application.

The project consists of three primary modules:

- Keyboard Event Capture Module
- SQLite Database Module
- Flask Monitoring Dashboard
- Tkinter Virtual Keyboard (Countermeasure)

Each module was tested independently and then integrated into a complete working system. The observations recorded during testing demonstrate that the project successfully achieves its intended educational objectives.

5.2 EXPERIMENTAL SETUP

The project was executed on a standalone Windows computer using Python 3.x with all required libraries installed.

Test Environment

Parameter	Configuration
Operating system	Windows 11
Programming Language	Python 3.x
IDE	Visual Studio Code
Database	SQLite
Browser	Google chrome
GUI Toolkit	TKinter
Web Framework	Flask

The application was executed locally without any network communication or remote deployment.

5.3 MODULE-WISE TESTING

Test 1 – Database Initialization

Objective-

To verify that the SQLite database is automatically created when the application starts.

Procedure-

1. Execute the application.
2. Observe whether the database file is created.
3. Verify the existence of the keystrokes table.

Expected Result-

Database should initialize automatically.

Actual Result-

Database was successfully created.

Status-

PASS 

Test 2 – Keyboard Event Capture

Objective-

To verify that keyboard events are detected.

Procedure-

1. Start the application.
2. Press multiple keys.
3. Observe database entries.

Expected Result-

Each key press should be stored.

Actual Result-

Keyboard events were successfully recorded and stored.

Status-

PASS 

Test 3 – Database Storage

Objective-

To verify that every captured key is stored together with its timestamp.

Procedure-

1. Press multiple characters.
2. Observe database records.

Expected Result-

Database should contain

- Timestamp
- Captured key

Actual Result-

All records were stored correctly.

Status-

PASS 

Test 4 – Flask Dashboard

Objective-

To verify the dashboard displays recorded entries.

Procedure-

1. Open <http://127.0.0.1:5000>
2. Observe displayed logs.

Expected Result-

Latest keyboard records should appear.

Actual Result-

Dashboard successfully displayed timestamped entries.

Status-

PASS 

Test 5 – Auto Refresh

Objective-

Verify automatic dashboard update.

Procedure-

1. Continue typing.
2. Observe dashboard.

Expected Result-

New records appear every three seconds.

Actual Result-

Dashboard updated successfully.

Status-

PASS 

Test 6 – Virtual Keyboard

Objective-

Verify operation of the Tkinter virtual keyboard.

Procedure-

Click

- Letters
- Numbers
- Space
- Backspace
- Clear

Expected Result-

Characters should appear inside the display field.

Actual Result-

All buttons functioned correctly.

Status

PASS 

5.4 SCREENSHOTS OF IMPLEMENTATION



Figure 5.1 Virtual Keyboard Window

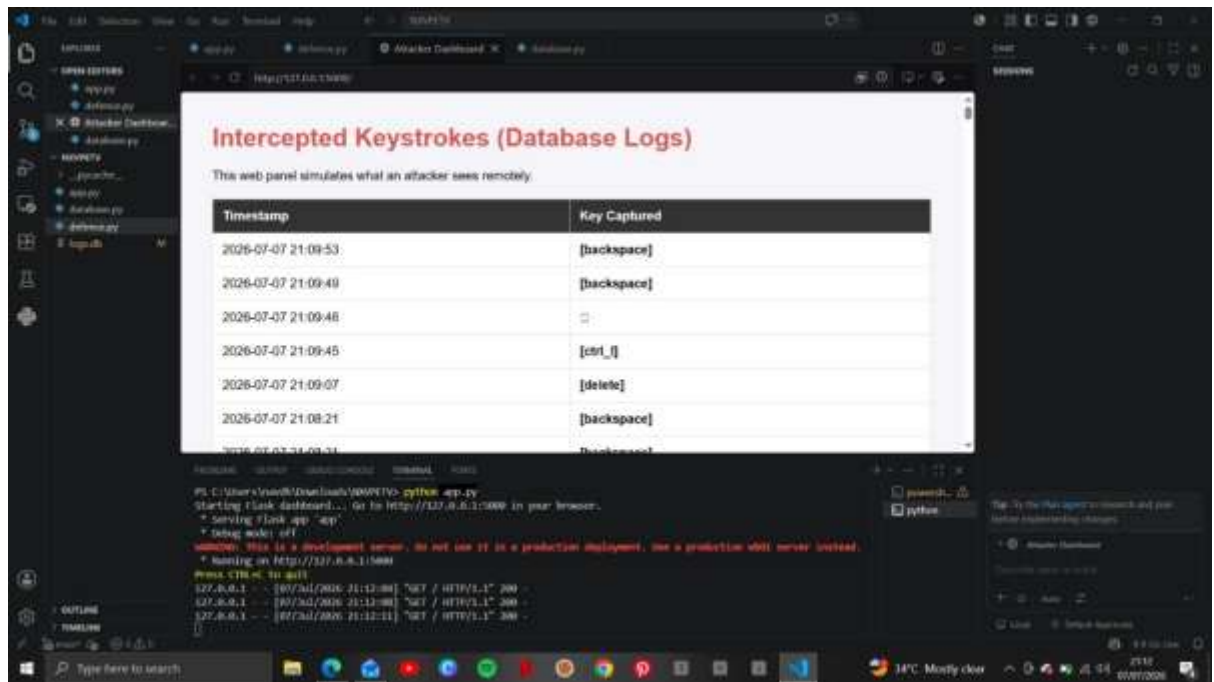


Figure 5.2 Captured Keystroke Logs

5.5 SAMPLE OUTPUT

Example of recorded database entries.

Timestamp	Key Captured
2026-06-15 10:30:01	A
2026-06-15 10:30:02	B
2026-06-15 10:30:04	[SPACE]
2026-06-15 10:30:08	C
2026-06-15 10:30:10	[ENTER]

5.6 FUNCTIONAL ANALYSIS

The integrated application performs its operations in the following sequence:

1. The SQLite database initializes.
2. A background thread starts the keyboard listener.

3. The Flask server launches.
4. Keyboard events are detected.
5. Captured events are stored.
6. The dashboard retrieves updated records.
7. Users can simultaneously interact with the virtual keyboard.

This sequence demonstrates successful integration of all project modules.

5.7 PERFORMANCE ANALYSIS

The application demonstrated satisfactory performance during testing.

Database Performance

SQLite insertion time remained low due to the lightweight database structure.

Dashboard Performance

Automatic page refresh occurred every three seconds without noticeable delay.

GUI Performance

The Tkinter interface remained responsive throughout testing.

Memory Usage

The application consumed relatively low system resources because it relied on lightweight Python libraries.

5.8 ADVANTAGES OBSERVED

During testing, several advantages were observed:

- Simple installation process.
- Fast database operations.

- Responsive web dashboard.
- Lightweight implementation.
- Easy-to-understand modular architecture.
- User-friendly graphical interface.
- Efficient integration of multiple Python libraries.

5.9 LIMITATIONS OBSERVED

Testing also revealed certain limitations.

- Local execution only.
- SQLite suitable only for small-scale demonstrations.
- No authentication for dashboard access.
- No process-monitoring component.
- No encrypted database.
- No alert notification mechanism.
- No behavioral anomaly analysis.

These limitations provide opportunities for future enhancement.

5.10 OBJECTIVE ACHIEVEMENT ANALYSIS

Objective	Achieved	Remarks
Demonstrate keyboard event capture	✓	Successfully implemented
Store captured events	✓	SQLite database
Display dashboard	✓	Flask application
Educational countermeasure	✓	Tkinter virtual keyboard
Local educational demonstration	✓	Successfully achieved

5.11 TEST CASE SUMMARY

Test Case	Result
Database Initialization	PASS
Keyboard Event Capture	PASS
SQLite Storage	PASS
Dashboard Display	PASS
Auto Refresh	PASS
Virtual Keyboard	PASS

5.12 INTERPRETATION OF RESULTS

The testing results indicate that the developed application successfully demonstrates the core concepts of software-based keystroke logging and local data visualization. Keyboard events were captured and stored reliably in the SQLite database, while the Flask dashboard accurately displayed the recorded entries. The Tkinter-based virtual keyboard functioned as intended, providing an alternative input interface for educational discussion.

The integration of these components demonstrates how multiple Python technologies can be combined to create an interactive cybersecurity learning environment. Although the project is intentionally limited to a local educational context, it effectively illustrates important concepts related to endpoint monitoring, data handling, and defensive awareness.

CHAPTER 6

6.1 DISCUSSION AND RESULTS

The implementation of the Keylogger & Countermeasure Demonstrator successfully fulfilled its primary objective of providing an educational platform for understanding software-based keystroke logging and basic defensive awareness. This chapter discusses

the outcomes of the project, evaluates its effectiveness, examines the significance of the results, and analyzes both the strengths and limitations of the developed system.

Unlike Chapter V, which focused on implementation and testing, this chapter interprets the observed behavior of the application from a cybersecurity perspective and relates the results to the project's objectives.

6.2 DISCUSSION OF PROJECT OUTCOMES

The project demonstrated that multiple Python technologies can be integrated effectively to create an educational cybersecurity application. The combination of keyboard-event handling, local database management, web visualization, and a graphical desktop interface provides learners with a practical understanding of how different software components interact within a cybersecurity context.

During implementation, the project successfully integrated:

- Python as the primary programming language.
- pynput for keyboard event handling.
- SQLite for local data storage.
- Flask for web-based visualization.
- Tkinter for the desktop-based countermeasure demonstration.
- Threading to allow simultaneous execution of background tasks and the web server.

The modular architecture also improved maintainability by separating data storage, user interaction, and dashboard functionality into independent modules.

6.3 ANALYSIS OF INDIVIDUAL MODULES

Keyboard Event Capture Module

The keyboard event listener functioned as expected by detecting keyboard events and converting them into readable text representations before storing them in the database.

Special keys such as Space and Enter were recorded using descriptive labels, improving the readability of stored records.

The implementation demonstrates the concept of keyboard-event interception in a controlled environment while remaining suitable for educational purposes.

Database Module

The SQLite database provided an efficient and lightweight solution for storing captured keyboard events.

During testing, the database successfully:

- Created the required table automatically.
- Stored timestamped keyboard records.
- Retrieved stored information without errors.

The simplicity of SQLite makes it appropriate for educational demonstrations because it requires minimal configuration while providing reliable relational data management.

Flask Dashboard

The Flask dashboard effectively displayed stored records retrieved from the database.

Automatic refreshing of the webpage ensured that recently stored entries became visible without requiring manual page reloads. This feature enhanced the educational value of the project by allowing users to observe how newly recorded information appears within the monitoring interface.

Countermeasure Module

The Tkinter virtual keyboard demonstrated an alternative method of entering characters using an on-screen interface.

Although simple in implementation, this module introduces learners to the concept of alternative input mechanisms that may reduce dependence on conventional physical

keyboard input in certain scenarios. The graphical interface also improves the usability and interactive nature of the project.

6.4 ACHIEVEMENT OF PROJECT OBJECTIVES

The objectives defined during the planning stage were evaluated against the final implementation.

Objective	Status	Remarks
Demonstrate keyboard-event handling	Achieved	Successfully implemented using pynput
Store captured data	Achieved	SQLite database used
Display stored records	Achieved	Flask dashboard implemented
Demonstrate defensive interface	Achieved	Tkinter virtual keyboard developed
Improve cybersecurity awareness	Achieved	Educational objectives fulfilled

6.5 DISCUSSION FROM A CYBERSECURITY PERSPECTIVE

From a cybersecurity perspective, the project highlights the importance of endpoint security by demonstrating that sensitive information may be exposed at the point of user input.

A significant observation from the implementation is that keyboard events are processed before network encryption occurs. This reinforces the principle that securing communication channels alone is insufficient if endpoint devices themselves are compromised.

The project also emphasizes that understanding offensive techniques is valuable for developing effective defensive strategies. By studying how keyboard-event monitoring

operates, cybersecurity students gain practical insight into the risks associated with credential theft and the need for secure endpoint practices.

6.6 STRENGTHS OF THE PROPOSED SYSTEM

Several strengths were observed during the implementation.

Educational Value

The project transforms theoretical cybersecurity concepts into an interactive learning experience.

Modular Design

Each functional component operates independently, making the project easier to understand, modify, and extend.

Lightweight Architecture

The application uses lightweight technologies such as SQLite and Flask, reducing hardware requirements while maintaining functionality.

User-Friendly Interface

The graphical user interface and monitoring dashboard simplify interaction and improve the overall learning experience.

Practical Demonstration

Students can directly observe how keyboard events are captured, stored, and displayed within a controlled environment.

6.7 LIMITATIONS OF THE DEVELOPED SYSTEM

Despite achieving its educational objectives, the project has several limitations.

Local Environment Only

The application is designed exclusively for standalone educational systems.

Limited Database Features

SQLite is suitable for small-scale educational projects but may not support enterprise-scale deployments.

No Authentication

The dashboard currently does not implement user authentication or role-based access control.

Basic Countermeasure

The Tkinter virtual keyboard provides only a conceptual demonstration and does not represent a comprehensive endpoint protection mechanism.

No Behavioral Detection

Although the project title refers to a countermeasure, the implementation does not currently include process monitoring, anomaly detection, or behavioral analysis.

No Alert Generation

The system does not notify users when suspicious activity occurs.

6.8 COMPARISON WITH EXISTING EDUCATIONAL APPROACHES

Existing Approach	Proposed Project
Mostly theoretical	Practical demonstration
Limited interaction	Interactive dashboard
No database implementation	SQLite integration
Minimal GUI	Tkinter interface
Static explanations	Real-time educational visualization

6.9 PRACTICAL APPLICATIONS

Although developed as an educational project, the concepts demonstrated have relevance in several areas.

- Cybersecurity laboratory demonstrations.
 - Academic teaching and learning.
 - Malware analysis training.
 - Endpoint security awareness sessions.
 - Ethical hacking education.
 - Software engineering demonstrations involving Python integration.
-

6.10 FUTURE IMPROVEMENTS

The current implementation provides several opportunities for enhancement.

Potential future developments include:

- Process-based behavioral detection.
- File integrity monitoring.
- Real-time security alerts.
- Dashboard authentication.
- Database encryption.
- Role-based access control.
- Machine learning-based anomaly detection.
- Email notification system.
- Cloud-based monitoring dashboard.
- SIEM integration.
- Multi-user support.
- Detailed audit logging.

These improvements would significantly increase the project's educational and technical value.

6.11 OVERALL EVALUATION

The project successfully demonstrates the fundamental concepts of software-based keyboard-event monitoring while integrating multiple Python technologies into a single educational platform.

The modular architecture, responsive interfaces, and lightweight implementation contribute to a system that is both functional and suitable for undergraduate cybersecurity education. The project effectively bridges theoretical concepts with practical implementation, enabling learners to better understand endpoint security concepts within a controlled environment.

CHAPTER 7

CONCLUSION

The Keylogger & Countermeasure Demonstrator was developed as an educational cybersecurity project to demonstrate the principles of software-based keyboard-event monitoring and introduce a simple defensive concept through an alternative input mechanism. The project integrates multiple Python technologies to provide a practical learning environment that bridges theoretical cybersecurity concepts with software implementation.

The successful completion of the project demonstrates how lightweight open-source technologies can be combined to create an interactive educational application that enhances understanding of endpoint security concepts while promoting ethical cybersecurity practices.

The Keylogger & Countermeasure Demonstrator successfully fulfills its purpose as an educational cybersecurity application. By integrating keyboard-event handling, local database storage, web-based visualization, and a graphical countermeasure interface, the

project demonstrates fundamental concepts related to endpoint security within a controlled environment.

The implementation provides a practical platform for understanding how keyboard-event monitoring operates while emphasizing ethical responsibility and cybersecurity awareness. Although intentionally limited in scope, the project establishes a strong foundation for future enhancements involving behavioral analysis, process monitoring, advanced endpoint detection, and intelligent security analytics.

Overall, the project demonstrates that effective cybersecurity education can be achieved through practical implementation, enabling students to connect theoretical concepts with real-world software development techniques while promoting responsible use of cybersecurity knowledge.

CHAPTER 8

8.1 RECOMMENDATIONS & FUTURE SCOPE

The Keylogger & Countermeasure Demonstrator successfully fulfills its primary objective of providing an educational platform for understanding software-based keyboard-event monitoring and endpoint security awareness. Although the developed application demonstrates the fundamental concepts effectively, cybersecurity is a rapidly evolving field that requires continuous improvement to address emerging threats and technological advancements.

This chapter presents recommendations for improving the current implementation and discusses future enhancements that could extend the educational and technical capabilities of the project. These suggestions aim to increase the practicality, scalability, and effectiveness of the system while maintaining its ethical and educational focus.

8.2 TECHNICAL RECOMMENDATIONS

Several technical improvements can be incorporated into future versions of the project.

Enhanced Database Management

The current implementation uses SQLite due to its simplicity and lightweight architecture. While SQLite is well suited for educational demonstrations, larger deployments could benefit from more robust database management systems such as:

- MySQL
- PostgreSQL
- Microsoft SQL Server

These databases provide improved scalability, concurrent user support, backup mechanisms, and enhanced security features.

Modular Code Expansion

Although the project is organized into separate modules, future versions may further improve maintainability by introducing:

- Object-Oriented Programming (OOP)
- Class-based architecture
- Configuration files
- Modular logging framework
- Exception handling modules

These improvements would simplify future development and maintenance.

Improved Dashboard Design

The Flask dashboard currently presents captured records using a simple HTML table. Future improvements may include:

- Responsive web design
- Search functionality

- Filter options
- Pagination
- Graphical statistics
- User-friendly navigation
- Dashboard themes (Light/Dark Mode)

These enhancements would improve usability and provide better visualization of recorded information.

8.3 SECURITY RECOMMENDATIONS

As cybersecurity projects should emphasize defensive principles, several security-oriented enhancements are recommended.

User Authentication

Future versions should include authentication mechanisms such as:

- Username and password login
- Multi-factor authentication
- Role-based access control
- Session management

This would prevent unauthorized users from accessing monitoring information.

Database Encryption

Although SQLite securely stores information locally, encrypting stored data would provide additional protection.

Possible encryption methods include:

- AES-256 Encryption
- SQLCipher
- Encrypted database files

This would reduce the risk of unauthorized access to stored records.

Secure Communication

If future versions include remote dashboards, communication should use:

- HTTPS
- TLS Encryption
- Secure Authentication Tokens

These technologies would protect transmitted information from interception.

8.4 COUNTERMEASURE IMPROVEMENTS

The current implementation demonstrates an educational virtual keyboard interface. Future work could expand the defensive capabilities of the project by incorporating additional endpoint protection concepts.

Possible enhancements include:

- Behavioral process monitoring.
- Detection of suspicious keyboard-hook activity.
- File integrity monitoring.
- Application reputation checking.
- Unauthorized process detection.
- Security notifications and alerts.

These features would improve the educational value of the project by introducing modern endpoint detection concepts without changing its primary educational focus.

8.5 ARTIFICIAL INTELLIGENCE INTEGRATION

Modern cybersecurity increasingly relies on Artificial Intelligence (AI) and Machine Learning (ML) for threat detection.

Future versions of the project could incorporate:

- Machine learning-based anomaly detection.
- Behavioral analysis of application activity.
- Classification of suspicious processes.
- Predictive threat analysis.
- Intelligent alert prioritization.

These additions would expose students to current cybersecurity research trends while enhancing the analytical capabilities of the system.

8.6 CLOUD INTEGRATION

Currently, the application operates entirely on a standalone local computer.

Future versions could integrate cloud technologies such as:

- Cloud database synchronization.
- Cloud dashboard access.
- Secure backup services.
- Remote administrative monitoring.
- Multi-device synchronization.

Cloud integration would improve scalability while maintaining centralized management of monitoring information.

8.7 USER INTERFACE ENHANCEMENTS

Several improvements can be introduced to improve user experience.

Suggested enhancements include:

- Modern graphical themes.
- Improved dashboard layout.

- Interactive charts.
- Animated notifications.
- Real-time activity indicators.
- Customizable keyboard layouts.
- Accessibility features for visually impaired users.

These modifications would make the application more interactive and easier to use.

8.8 PERFORMANCE OPTIMIZATION

Future versions could improve system performance through:

- Optimized database queries.
- Background task scheduling.
- Reduced memory consumption.
- Faster dashboard updates.
- Improved multithreading.
- Efficient resource management.

These optimizations would support smoother execution during extended demonstrations.

8.9 RESEARCH OPPORTUNITIES

The project opens several opportunities for future academic research.

Potential research directions include:

- Endpoint Detection and Response (EDR) systems.
- Host Intrusion Detection Systems (HIDS).
- User behavior analytics.
- Secure input mechanisms.
- Malware detection techniques.
- Cybersecurity awareness training systems.

- Human-computer interaction in secure environments.

These topics provide valuable directions for undergraduate and postgraduate cybersecurity research.

8.10 INDUSTRIAL APPLICATIONS

Although developed as an educational demonstration, the concepts presented in the project have practical relevance in several domains.

Potential application areas include:

- Cybersecurity laboratories.
 - Academic institutions.
 - Digital forensics training.
 - Malware analysis education.
 - Security awareness workshops.
 - Ethical hacking courses.
 - Software engineering education.
 - Research institutions.
-

8.11 OVERALL RECOMMENDATIONS

Based on the implementation and evaluation of the project, the following recommendations are proposed:

1. Continue developing the project as an educational cybersecurity platform.
2. Introduce stronger authentication and database security.
3. Expand defensive capabilities through behavioral monitoring.
4. Improve the graphical dashboard using modern web technologies.
5. Incorporate machine learning for anomaly detection.
6. Enhance scalability using enterprise database systems.
7. Improve reporting and visualization features.

8. Maintain ethical guidelines for all future developments.
-

8.12 FUTURE SCOPE

The future scope of the Keylogger & Countermeasure Demonstrator extends beyond its current educational implementation.

Future versions may incorporate:

- Intelligent endpoint monitoring.
- Advanced behavioral analytics.
- Secure event logging.
- Automated threat detection.
- Cloud-based dashboards.
- Multi-user support.
- Cross-platform compatibility.
- Mobile monitoring applications.
- Integration with Security Information and Event Management (SIEM) systems.
- Real-time cybersecurity alert generation.

These enhancements would significantly improve both the educational and practical value of the project while introducing students to modern cybersecurity technologies.

APPENDIX

Appendix A- database.py

```
import sqlite3

from datetime import datetime

DB_NAME = "logs.db"

def init_db():
    """Creates the database and table if they do not exist."""
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS keystrokes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp TEXT,
            key_pressed TEXT
        )
    """)
    conn.commit()
    conn.close()

def save_key(key_str):
    """Saves a single keypress into the database."""
    conn = sqlite3.connect(DB_NAME)
    cursor = conn.cursor()
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    cursor.execute("INSERT INTO keystrokes (timestamp, key_pressed) VALUES (?, ?)",
        (now, key_str))
    conn.commit()
    conn.close()
```

```
def get_all_keys():  
    """Retrieves all logged keys to display on the Flask web dashboard."""  
    conn = sqlite3.connect(DB_NAME)  
    cursor = conn.cursor()  
    cursor.execute("SELECT timestamp, key_pressed FROM keystrokes ORDER BY id  
DESC")  
    rows = cursor.fetchall()  
    conn.close()  
    return rows  
  
init_db()
```

Appendix B- app.py

```
from flask import Flask, render_template_string
from pynput import keyboard
import threading
import database

app = Flask(__name__)

def on_press(key):
    try:
        key_name = key.char
    except AttributeError:
        if key == keyboard.Key.space:
            key_name = " [SPACE] "
        elif key == keyboard.Key.enter:
            key_name = " [ENTER] "
        else:
            key_name = f" [{key.name}] "

    # Save the key directly to our SQLite database
    database.save_key(key_name)

def start_keylogger():
    with keyboard.Listener(on_press=on_press) as listener:
        listener.join()

HTML_TEMPLATE = """
<!DOCTYPE html>
<html>
<head>
```

```

<title>Attacker Dashboard</title>

<meta http-equiv="refresh" content="3"> <!-- Auto-refreshes page every 3 seconds -->

<style>
  body { font-family: Arial, sans-serif; margin: 40px; background-color: #f4f4f9; }
  h1 { color: #d9534f; }
  table { width: 100%; border-collapse: collapse; margin-top: 20px; background: white; }
  th, td { padding: 12px; border: 1px solid #ddd; text-align: left; }
  th { background-color: #333; color: white; }
</style>
</head>
<body>
  <h1>Intercepted Keystrokes (Database Logs)</h1>
  <p>This web panel simulates what an attacker sees remotely.</p>
  <table>
    <tr>
      <th>Timestamp</th>
      <th>Key Captured</th>
    </tr>
    {% for row in logs %}
      <tr>
        <td>{{ row[0] }}</td>
        <td><strong>{{ row[1] }}</strong></td>
      </tr>
    {% endfor %}
  </table>
</body>
</html>

```

```

@app.route('/')

```



```

def index():
    logs = database.get_all_keys()
    return render_template_string(HTML_TEMPLATE, logs=logs)

if __name__ == '__main__':
    logger_thread = threading.Thread(target=start_keylogger, daemon=True)
    logger_thread.start()

    print("Starting Flask dashboard... Go to http://127.0.0.1:5000 in your browser.")
    app.run(port=5000, debug=False)

```

Appendix C- defence.py

```

import tkinter as tk

def press_key(letter):
    """Simulates a key input inside the secure app text box."""
    current = display_var.get()

    if letter == "Space":
        display_var.set(current + " ")
    elif letter == "Clear":
        display_var.set("")

```

```

elif letter == "Backspace":
    display_var.set(current[:-1])
else:
    display_var.set(current + letter)

root = tk.Tk()
root.title("Secure Virtual Keyboard")
root.geometry("650x330") # Made wider to easily fit a full keyboard layout
root.attributes("-topmost", True) # Keeps the defense tool visible on top

display_var = tk.StringVar()

label = tk.Label(
    root,
    text="Countermeasure: Virtual Typing Panel\n(Bypasses pynput hooks)",
    fg="green",
    font=("Arial", 11, "bold")
)
label.pack(pady=5)

entry = tk.Entry(root, textvariable=display_var, font=("Arial", 14), justify='center', width=45)
entry.pack(pady=10)

frame = tk.Frame(root)
frame.pack(pady=5)

keyboard_rows = [
    ['1', '2', '3', '4', '5', '6', '7', '8', '9', '0', '-', '='],
    ['Q', 'W', 'E', 'R', 'T', 'Y', 'U', 'I', 'O', 'P', '[', ']'],
    ['A', 'S', 'D', 'F', 'G', 'H', 'J', 'K', 'L', ';', "'", 'Backspace'],

```

```

['Z', 'X', 'C', 'V', 'B', 'N', 'M', ',', '.', '/', 'Space', 'Clear']
]

for row_index, row_content in enumerate(keyboard_rows):
    for col_index, btn_text in enumerate(row_content):
        # Using a default argument inside lambda to correctly bind the text value
        action = lambda x=btn_text: press_key(x)

        if btn_text in ["Space", "Clear", "Backspace"]:
            btn = tk.Button(
                frame,
                text=btn_text,
                width=10,
                command=action,
                bg="#d3d3d3",
                activebackground="#b8b8b8",
                font=("Arial", 9, "bold")
            )
        else:
            btn = tk.Button(
                frame,
                text=btn_text,
                width=4,
                height=2,
                command=action,
                font=("Arial", 10)
            )
        btn.grid(row=row_index, column=col_index, padx=3, pady=3)

root.mainloop()

```

BIBLIOGRAPHY

1. **Stallings, William.** *Computer Security: Principles and Practice* (4th Edition). Pearson Education.
2. **SANS Institute.** *Endpoint Security Resources*.
<https://www.sans.org>
3. **Python Software Foundation.** *Python 3 Documentation*.
<https://docs.python.org/3/>
4. **Flask Documentation.** *Flask Web Framework Documentation*.
<https://flask.palletsprojects.com/>
5. **SQLite Documentation.** *Official SQLite Documentation*.
<https://www.sqlite.org/docs.html>
6. **TkDocs.** *Tkinter GUI Programming*.
<https://tkdocs.com/>
7. **pynput Documentation.** *Keyboard and Mouse Listener Library*.
<https://pynput.readthedocs.io/>